```
  g.test_framework  :test_unit, fixture: false
  g.stylesheets     false
  g.javascripts     false
  g.helper          :my_helper
end
```

and see it in action when invoking the generator:

```
$ bin/rails generate scaffold Article body:text
      [...]
      invoke    my_helper
      create      app/helpers/articles_helper.rb
```

We can notice on the output that our new helper was invoked instead of the Rails default. However one thing is missing, which is tests for our new generator and to do that, we are going to reuse old helpers test generators.

Since Rails 3.0, this is easy to do due to the hooks concept. Our new helper does not need to be focused in one specific test framework, it can simply provide a hook and a test framework just needs to implement this hook in order to be compatible.

To do that, we can change the generator this way:

```
# lib/generators/rails/my_helper/my_helper_generator.rb
class Rails::MyHelperGenerator < Rails::Generators::NamedBase
  def create_helper_file
    create_file "app/helpers/#{file_name}_helper.rb", <<-FILE
module #{class_name}Helper
  attr_reader :#{plural_name}, :#{plural_name.singularize}
end
    FILE
  end

  hook_for :test_framework
end
```

Now, when the helper generator is invoked and TestUnit is configured as the test framework, it will try to invoke both `Rails::TestUnitGenerator` and `TestUnit::MyHelperGenerator`. Since none of those are defined, we can tell our generator to invoke `TestUnit::Generators::HelperGenerator`instead, which is defined since it's a Rails generator. To do that, we just need to add:

```
# Search for :helper instead of :my_helper
hook_for :test_framework, as: :helper
```

And now you can re-run scaffold for another resource and see it generating tests as well!

6 Customizing Your Workflow by Changing Generators Templates

In the step above we simply wanted to add a line to the generated helper, without adding any extra functionality. There is a simpler way to do that, and it's by replacing the templates of already existing generators, in that case `Rails::Generators::HelperGenerator`.

In Rails 3.0 and above, generators don't just look in the source root for templates, they also search for templates in other paths. And one of them is `lib/templates`. Since we want to customize `Rails::Generators::HelperGenerator`, we can do that by simply making a template copy inside `lib/templates/rails/helper` with the name `helper.rb`. So let's create that file with the following content:

```
module <%= class_name %>Helper
  attr_reader :<%= plural_name %>, :<%= plural_name.singularize %>
```